

JUnit4: Property based Testing & Approval Testing

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation: Mehr als nur Beispieltests

- Bisher: klassische Unit-Tests mit JUnit (`@Test`, `assertEquals`, ...)
- Problem 1: Komplexe/umfangreiche Outputs sind schwer “per Hand” zu vergleichen
- Problem 2: Einzelne Beispieltests decken nur wenige Eingaben ab

- Idee: Zwei Ergänzungen
 - Approval Testing
 - Property-Based Testing (PBT)

Approval Testing: Idee und Einsatzgebiete

```
@Test
void report_looks_as_expected() {
    // given
    List<Order> orders = List.of(...); // !!!!
    String expected = "..."; // ???!

    // when
    String report = OrderReportGenerator.generateReport(orders);

    // then
    assertEquals(expected, report);
}
```

- Grundidee:
 - Einmal gültigen Output als “Goldstandard” abspeichern (approved)
 - Zukünftig generierten Output automatisch dagegen vergleichen

Beispiel: Order-Report mit manuellen Approval-Tests

```
public record Order(String id, String customer, List<String> items) {}

public class OrderReportGenerator {
    public static String generateReport(List<Order> orders) {
        var sb = new StringBuilder();
        sb.append("=== ORDER REPORT ===\n");
        orders.forEach( o -> {
            sb.append("Order: ").append(o.id()).append("\n");
            sb.append("Customer: ").append(o.customer()).append("\n");
            sb.append("Items:\n");
            o.items().forEach(item -> sb.append(" - ").append(item).append("\n"));
            sb.append("\n");
        });
        sb.append("Total orders: ").append(orders.size()).append("\n");
        return sb.toString();
    }
}
```

Erster Lauf mit leerem "expected"-String:

```
class OrderReportGeneratorTest {

    @Test
    void report_looks_as_expected() {
        // given
        List<Order> orders = List.of(
            new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
            new Order("B-002", "Bob", List.of("Laptop")),
            new Order("C-003", "Charlie", List.of("HDMI Cable", "USB Hub")));
        // 1. Test absichtlich fehlschlagen lassen, 2. Ausgabe übernehmen
        String expected = "..."; // ???!

        // when
        String report = OrderReportGenerator.generateReport(orders);

        // then
        assertEquals(expected, report);
    }
}
```

Nach dem ersten Lauf Ergänzen des “expected”-Strings zu:

```
class OrderReportGeneratorTest {

    @Test
    void report_looks_as_expected() {
        // given
        List<Order> orders = List.of(
            new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
            new Order("B-002", "Bob", List.of("Laptop")),
            new Order("C-003", "Charlie", List.of("HDMI Cable", "USB Hub")));
        String expected = ""
            === ORDER REPORT ===
            Order: A-001
            Customer: Alice
            Items:
            - Keyboard
            - Mouse

            Order: B-002
            Customer: Bob
            Items:
```

Beispiel: Order-Report mit Bibliothek “ApprovalTests”

```
class OrderReportGeneratorTest {

    @Test
    void report_looks_as_expected() {
        // given
        List<Order> orders = List.of(
            new Order("A-001", "Alice", List.of("Keyboard", "Mouse")),
            new Order("B-002", "Bob", List.of("Laptop")),
            new Order("C-003", "Charlie", List.of("HDMI Cable", "USB Hub")));

        // when
        String report = OrderReportGenerator.generateReport(orders);

        // then
        // ApprovalTests.Java: vergleicht gegen <TESTCLASS>.<TESTMETHOD>.approved.txt
        Approvals.verify(report);
    }
}
```

- Approved-Dateien werden im Repository versioniert
- Diffs der Approved-Files zeigen Verhaltensänderungen im Code-Review
- Workflow bei legitimer Änderung:
 - Test schlägt fehl -> Diff prüfen -> neue Version als “approved” übernehmen

Property-Based Testing: Grundidee

- Klassischer Test:
 - “Für Eingabe X erwarte ich Ergebnis Y”
- Property-Based Test:
 - “Für alle Eingaben mit Eigenschaft E muss Eigenschaft P gelten”
- Ein Framework (z.B. jqwik) generiert viele Eingaben automatisch
- Fokus: Allgemeine Eigenschaften / Invarianten statt einzelner Beispiele

Anknüpfung: Äquivalenzklassenanalyse

- Äquivalenzklassenanalyse:
 - Eingaberaum in Bereiche mit ähnlichem Verhalten aufteilen
 - Je Klasse einige repräsentative Testfälle + Grenzwerte wählen
- Property-Based Testing (PBT):
 - Nutzt dieselben Bereiche/Eigenschaften
 - Aber: Framework erzeugt viele Werte innerhalb der Bereiche
- Kein “doppelt genäht”, sondern:
 - Äquivalenzklassen = Denkwerkzeug
 - PBT = automatisierte Stichproben über diese Klassen

Durchgängiges Beispiel: Steuerfunktion - Spezifikation

- Funktion: `calculateTax(int income)`
 - Einkommen in ganzen Euro (`income >= 0`)
 - Rückgabe: Steuerbetrag (ganze Euro, abgerundet)
- Steuerregeln (vereinfacht):
 - $< 10\,000$: 0 %
 - $10\,000 - 20\,000$: 10 % auf den Teil über 10 000
 - $20\,000 - 50\,000$: zusätzlich 20 % auf den Teil über 20 000
 - $> 50\,000$: zusätzlich 30 % auf den Teil über 50 000
- Negative Einkünfte: `IllegalArgumentException`

Steuerfunktion: Äquivalenzklassen & Grenzwerte

- Äquivalenzklassen:
 - E1: `income < 0` (ungültig)
 - E2: `0 <= income < 10000`
 - E3: `10000 <= income <= 20000`
 - E4: `20000 < income <= 50000`
 - E5: `income > 50000`
- Wichtige Grenzwerte:
 - Rund um 0: `-1`, `0`, `1`
 - Rund um 10000: `9999`, `10000`, `10001`
 - Rund um 20000: `19999`, `20000`, `20001`
 - Rund um 50000: `49999`, `50000`, `50001`

Klassische JUnit-Tests: Beispiele aus den Klassen

- Für jede Äquivalenzklasse einige repräsentative Werte testen
- Zusätzlich Grenzfälle explizit prüfen
- Negative Eingaben: Exception erwarten

```
// E1: Ungültige Eingabe
@Test
void negative_income_throws_exception() {
    assertThrows(IllegalArgumentException.class, () -> TaxCalculator.calculateTax(-1));
}

// E2: 0 <= income < 10_000
@Test
void income_below_10k_is_tax_free() {
    assertEquals(0, TaxCalculator.calculateTax(0));
    assertEquals(0, TaxCalculator.calculateTax(5_000));
    assertEquals(0, TaxCalculator.calculateTax(9_999));
}

// ...
```

Ziel-Eigenschaft: Für alle gültigen Einkommen gilt: Steuerbetrag ≥ 0

```
class TaxCalculatorProperties {  
    @Property  
    void tax_is_never_negative(@ForAll @IntRange(min = 0, max = 1_000_000) int income) {  
        int tax = TaxCalculator.calculateTax(income);  
        assertTrue(tax >= 0);  
    }  
}
```

jqwik: Monotonie-Eigenschaft für die Steuer

- Intuition: Wenn Einkommen steigt, darf Steuer nicht sinken
- Formale Property: Für alle a, b mit $a \leq b$ gilt: $\text{tax}(a) \leq \text{tax}(b)$

```
@Property
void tax_is_monotone_non_decreasing(
    @ForAll @IntRange(min = 0, max = 1_000_000) int a,
    @ForAll @IntRange(min = 0, max = 1_000_000) int b) {

    int lower = Math.min(a, b);
    int higher = Math.max(a, b);

    int taxLower = TaxCalculator.calculateTax(lower);
    int taxHigher = TaxCalculator.calculateTax(higher);

    assertTrue(taxLower <= taxHigher);
}
```

jqwik: Verbindung zu Äquivalenzklassen (Bracket-Property)

- Idee: Innerhalb einer Steuerklasse (z.B. 10000 - 20000) gilt ein linearer Zusammenhang
- Property-Beispiel: Innerhalb 10000 - 20000 steigt die Steuer mit 10 % der Einkommensdifferenz

```
@Property
void within_bracket_10k_to_20k_tax_grows_with_roughly_10_percent(
    @ForAll @IntRange(min = 10_000, max = 19_000) int base,
    @ForAll @IntRange(min = 0, max = 1_000) int delta) {
    int income1 = base; int income2 = base + delta;
    if (income2 > 20_000) income2 = 20_000;

    int tax1 = TaxCalculator.calculateTax(income1);
    int tax2 = TaxCalculator.calculateTax(income2);
    int diffIncome = income2 - income1;
    int diffTax = tax2 - tax1;
    int expected = (int) Math.floor(0.10 * diffIncome);

    // Wegen Rundung erlauben wir +/- 1 Euro Toleranz
    assertTrue(diffTax >= expected - 1 && diffTax <= expected + 1);
}
```


- Klassische Unit-Tests:
 - Dokumentieren Verhalten an Beispielen
 - Nutzen Äquivalenzklassen & Grenzwerte manuell
- Approval Testing:
 - Erleichtert Tests komplexer Outputs (“Goldstandard”)
 - Passt gut zu Refactoring & Code-Review
- Property-Based Testing:
 - Formuliert allgemeine Eigenschaften über alle Eingaben
 - Nutzt Äquivalenzklassen als Denkgrundlage, generiert aber viele Fälle automatisch



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

